Multi-Agent Reinforcement Learning for America's Cup Regattas

Francesco Maria Fuligni
Mattia Furini
Mohamed Samir Haffoudhi
Daniele Silvestri

June 3, 2026

Abstract

This paper presents a Multi-Agent Reinforcement Learning (MARL) framework for a two-dimensional simulator of America's Cup regattas. The project extends a didactic reinforcement learning environment applied to sailing [1] into a competitive PettingZoo `ParallelEnv` [3], in which two autonomous vessels learn continuous rudder and sail trim control in the presence of stochastic wind, polar performance constraints, and hybrid dynamics between displacement and foiling navigation. The agents are trained with the PPO (Proximal Policy Optimization) algorithm [2] via Stable-Baselines3 [4] and SuperSuit vectorization. The available evidence consists of the trained model artifact, rendered trajectories, and training plots exported for mean episodic reward and mean VMG (Velocity Made Good).

1 Introduction

Autonomous sailing is a sequential decision-making problem in which control, navigation, and environmental adaptation are tightly coupled. A sailing vessel cannot move freely in arbitrary directions: its achievable speed is constrained by the wind angle, the aerodynamic efficiency of the sails, hydrodynamic resistance, and, in modern high-performance craft, the transition between displacement and foiling navigation. America's Cup regattas add a further competitive layer: each vessel must not only optimize its progress along the course, but also react to the opponent while respecting tactical constraints and racing rules.

This project implements a simplified yet technically rich 2D simulator of an America's Cup match race. The environment contains two agents, `red_boat` and `blue_boat`, a rectangular racing field, an upwind/downwind course with mandatory gates, a stochastic wind field, continuous rudder and sail trim actions, and a dense reward function designed to incentivize progress, VMG, trim efficiency, foil stability, and safe tactical interactions.

The work is conceptually linked to the course lectures [1], in which a single sailing vessel learns to reach a waypoint in a simplified Gymnasium environment, using state variables, actions, rewards, polar behavior, and basic VMG. The present project generalizes that didactic setup along four dimensions: it moves from a single-agent task to a competitive multi-agent environment, from simplified/discrete steering to continuous control, from a basic physical model to a hybrid displacement/foil polar model, and from a fixed or simple wind model to stochastic spatio-temporal wind dynamics.

2 Background

2.1 Notation and Agent–Environment Interaction

Following the notation used in the course notes [1], random variables are denoted with uppercase letters, such as S_t, A_t, and R_t, while their realizations are denoted with lowercase letters, such as s_t, a_t, and r_t. Additionally, $\Delta(X)$ denotes the set of all probability distributions over a finite set

X. Expected values are computed with respect to the probability distribution jointly induced by the policy and the environment dynamics.

In Reinforcement Learning, an agent interacts with the environment at discrete time steps. At time t, the agent observes a state S_t, selects an action A_t, receives a reward R_t, and the environment transitions to a new state S_{t+1}. The interaction can be written as

$$S_t \xrightarrow{A_t} (S_{t+1}, R_t). \tag{1}$$

In stochastic environments, the next state and reward are not deterministic functions of the current state and action. They are sampled from the distributions induced by the environment:

$$S_{t+1} \sim P(\cdot \mid S_t, A_t), \qquad R_t \sim R(\cdot \mid S_t, A_t). \tag{2}$$

2.2 Markov Decision Processes

The Markov property states that the future depends on the present state and not on the entire history:

$$\Pr(X_{t+1} = j \mid X_t = i, X_{t-1}, \ldots, X_0) = \Pr(X_{t+1} = j \mid X_t = i). \tag{3}$$

An MDP formalizes a sequential decision-making problem as a tuple

$$M = (\mathcal{S}, \mathcal{A}, P, r, \mu, \gamma), \tag{4}$$

where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P(s' \mid s, a)$ is the transition probability, $r(s, a)$ is the reward function, μ is the initial state distribution, and $\gamma \in [0, 1]$ is the discount factor.

In the current simulator, each vessel observes only a feature vector derived from the full simulator state. The implementation is therefore better interpreted as a partially observed multi-agent control problem, but the learning interface follows the standard MDP formulation at the level of each agent's observations and actions.

2.3 Policies, Trajectories, and Return

A policy defines how actions are selected from states. A deterministic policy has the form

$$A_t = \pi(S_t), \tag{5}$$

while a stochastic policy samples an action according to a conditional distribution:

$$A_t \sim \pi(\cdot \mid S_t). \tag{6}$$

For a finite horizon T, a trajectory can be written as

$$\tau = (s_0, a_0, s_1, a_1, \ldots, s_T, a_T). \tag{7}$$

Under a stochastic policy π, the probability of such a trajectory is

$$p(\tau) = p(s_0) \prod_{t=0}^{T} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t). \tag{8}$$

The goal of reinforcement learning is not to maximize immediate reward, but cumulative future reward. Without discounting, the sum of rewards $G_t = R_t + R_{t+1} + \cdots$ has no upper bound, which can lead to exploding gradients during optimization. To address this, we introduce a discount factor $\gamma \in [0, 1]$ that controls how much the agent prioritizes immediate versus future rewards:

$$G_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \gamma^3 R_{t+3} + \cdots = \sum_{k=0}^{\infty} \gamma^k R_{t+k}. \tag{9}$$

If the reward is constant ($R_t = R$), this simplifies to a geometric series:

$$G_t = \frac{R}{1 - \gamma}. \tag{10}$$

Depending on the starting state assumption, the objective function can be defined either by fixing the initial state or by taking the expectation over the initial state distribution μ:

$$J(\pi) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \right] \tag{11}$$

or

$$J(\pi) = \mathbb{E}_{s_0 \sim \mu, \pi} \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, A_t) \right]. \tag{12}$$

We need to find an optimal policy $\pi^\star$ that maximizes $J(\pi)$.

2.4 Value Functions and the Bellman Equation

The state value function measures the expected return starting from state s following policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s \right]. \tag{13}$$

The action value function measures the expected return when first executing action a in state s and then following policy π:

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t R_t \mid S_0 = s, A_0 = a \right]. \tag{14}$$

The two functions are related by the average of action values under the policy:

$$V^\pi(s) = \sum_a \pi(a \mid s) Q^\pi(s, a). \tag{15}$$

The Bellman expectation equation expresses the recursive structure of value:

$$V^\pi(s) = \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right]. \tag{16}$$

Equivalently, the action value function satisfies

$$Q^\pi(s, a) = r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s'). \tag{17}$$

The Bellman expectation equation can also be written in operator form:

$$(T_\pi V)(s) = \sum_a \pi(a \mid s) \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V(s') \right], \tag{18}$$

where the true value function V^π is the unique fixed point of this operator T_π.

2.5 Optimality

We aim to learn a policy that achieves optimal performance. The optimal state value function $V^\star(s)$ and optimal action value function $Q^\star(s, a)$ are defined as:

$$V^\star(s) = \sup_\pi V^\pi(s), \qquad Q^\star(s, a) = \sup_\pi Q^\pi(s, a). \tag{19}$$

An improved policy π' can be obtained by choosing actions greedily with respect to the state value function:

$$\pi'(s) \in \arg\max_a \left[r(s, a) + \gamma \sum_{s'} P(s' \mid s, a) V^\pi(s') \right], \tag{20}$$

satisfying $V^{\pi'} \geq V^\pi$ for all states.

2.6 Model-Free Learning, Function Approximation, and PPO

In model-free reinforcement learning, the transition model P and the reward model r are not assumed to be analytically known. The agent instead learns from sampled transitions (S_t, A_t, R_t, S_{t+1}). With function approximation, value functions or policies are represented by parameterized models, typically neural networks (for a comprehensive introduction to training neural networks, see [5]). The temporal difference error for a value approximation V_θ is

$$\delta_t = R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t). \tag{21}$$

Using gradient descent with learning rate α, we update the weights as:

$$\theta \leftarrow \theta + \alpha \delta_t \nabla_\theta V_\theta(S_t), \tag{22}$$

and the corresponding quadratic loss is

$$L(\theta) = \frac{1}{2} \left(R_t + \gamma V_\theta(S_{t+1}) - V_\theta(S_t) \right)^2 . \tag{23}$$

The same principles apply to the parameterization of the action value function $Q_\theta(s, a)$.

The America's Cup simulator has continuous observations, continuous actions, and nonlinear transition dynamics induced by wind, polar curves, foil state, and multi-agent interaction. For this reason, the implementation uses PPO (Proximal Policy Optimization) [2], provided by Stable-Baselines3 [4], as a practical model-free policy gradient method suited to continuous control. PPO is not used here to introduce additional theoretical machinery beyond the course notes [1]; it is rather the concrete algorithmic tool that maps the function approximation perspective into a trainable continuous control policy.

3 Methodology

3.1 Race Course and Multi-Agent Formulation

The environment is implemented as a PettingZoo `ParallelEnv` [3] with two agents:

$$\mathcal{I} = \{\texttt{red\_boat}, \texttt{blue\_boat}\}. \tag{24}$$

The field has width $W = 1900$ m and length $L = 4100$ m. The top gate is positioned at $y = 3900$ m, the bottom gate at $y = 200$ m, and the gate width is 300 m. Each episode has a maximum duration of 1800 simulation steps with $\Delta t = 1$ s.

The race is decomposed into sequential legs. The vessels start below the bottom gate, cross the starting gate, sail upwind toward the top gate, perform a rounding maneuver, and then sail downwind toward the bottom gate. Gate violations, course violations, collisions, and excessive rotations can terminate the episode with severe penalties.

3.2 Observation Space

Each agent receives a normalized continuous observation vector

$$o_t^i \in [-1, 1]^{23}. \tag{25}$$

The vector contains normalized position, speed, heading, relative wind angle, local wind speed, distances and bearings to the active gate marks, current rudder and trim state, foil state, active foil side, leg indicator, and opponent-relative features. With frame stacking configured to 4, the effective policy input is

$$\tilde{o}_t^i = (o_{t-3}^i, o_{t-2}^i, o_{t-1}^i, o_t^i) \in [-1, 1]^{92}. \tag{26}$$

Feature group	Observation components
Boat kinematics	x/W, y/L, speed divided by 50, and sin / cos encoding of heading.
Wind and controls	sin / cos encoding of local relative wind angle, local wind speed divided by 30, rudder, sail trim mapped to $[-1, 1]$, foil flag, and active foil side.
Course geometry	Normalized distances and relative bearings to left and right marks, plus a signed leg indicator.
Opponent state	Opponent relative position, normalized distance to opponent, and speed advantage.

Table 1: Observation structure used by each agent before frame stacking.

3.3 Action Space and Kinematics

The action space is two-dimensional and continuous:

$$a_t^i = (u_t^i, z_t^i) \in [-1, 1]^2, \tag{27}$$

where u_t^i is the rudder command and z_t^i is the sail trim command. The trim command is mapped to a physical trim level via

$$\tau_t = \text{clip}\left(\frac{z_t + 1}{2}, 0, 1\right). \tag{28}$$

The heading evolves according to a speed-dependent turn rate. Let v_t be the current speed in knots, $v_{\max} = 50$, $\rho_{\min} = 0.12$, and $\omega_{\max} = 25^\circ$. The effective turning factor is

$$\rho_t = \rho_{\min} + (1 - \rho_{\min})\frac{v_t}{v_{\max}}, \tag{29}$$

and the heading update is

$$\psi_{t+1} = (\psi_t + u_t \rho_t \omega_{\max} \Delta t) \bmod 2\pi. \tag{30}$$

The boat position is updated using the simulated speed converted from knots to meters per second:

$$p_{t+1} = p_t + 0.514\, v_{t+1} \Delta t \begin{bmatrix} \cos\psi_{t+1} \\ \sin\psi_{t+1} \end{bmatrix}. \tag{31}$$

3.4 Stochastic Wind Field

The wind model contains a global base wind and a spatial perturbation grid. At reset, the base wind direction is initialized near a north-to-south flow, with a random angular offset, and the base wind speed is sampled from $[15, 22]$ knots. At each step,

$$\Theta_{t+1} = (\Theta_t + \varepsilon_t^\Theta) \bmod 2\pi, \qquad \varepsilon_t^\Theta \sim \mathcal{U}(-0.02, 0.02), \tag{32}$$

$$U_{t+1} = \text{clip}(U_t + \varepsilon_t^U, 15, 22), \qquad \varepsilon_t^U \sim \mathcal{U}(-0.3, 0.3). \tag{33}$$

Local gusts are represented by a 10×10 grid. For each cell (m, n), the direction and speed perturbations evolve as

$$D_{t+1}^{m,n} = \text{clip}\left(0.95 D_t^{m,n} + \eta_t^{D,m,n}, -2.5(0.15), 2.5(0.15)\right), \tag{34}$$

$$S_{t+1}^{m,n} = \text{clip}\left(0.95 S_t^{m,n} + \eta_t^{S,m,n}, -2.5(2.0), 2.5(2.0)\right), \tag{35}$$

where $\eta_t^{D,m,n} \sim \mathcal{N}(0, 0.015^2)$ and $\eta_t^{S,m,n} \sim \mathcal{N}(0, 0.2^2)$. The local wind experienced by the boat is obtained by bilinear interpolation of the four nearest grid cells and then added to the base wind.

3.5 Polar Speed Model and Foil Dynamics

The code computes a relative wind angle $\phi_t \in [0, 180]$ degrees from the local wind direction and the boat heading. The theoretical polar speed is

$$v_t^{\text{base}} = \min(q_{\kappa_t}(\phi_t) U_t^{\text{local}}, 50), \tag{36}$$

where $\kappa_t \in \{D, F\}$ denotes displacement or foiling mode. In foiling mode, the multiplier is

$$q_F(\phi) = \begin{cases} 0, & 0 \le \phi < 45, \\ 1.0 + 0.15(\phi - 45), & 45 \le \phi < 55, \\ 2.5 + 0.024(\phi - 55), & 55 \le \phi < 100, \\ 3.6 + 0.015(\phi - 100), & 100 \le \phi < 140, \\ 4.2 - 0.03(\phi - 140), & 140 \le \phi < 170, \\ 3.3 - 0.25(\phi - 170), & 170 \le \phi \le 180. \end{cases} \tag{37}$$

In displacement mode, the multiplier is

$$q_D(\phi) = \begin{cases} 0, & 0 \le \phi < 35, \\ 0.3 + 0.03(\phi - 35), & 35 \le \phi < 50, \\ 0.75 + 0.015(\phi - 50), & 50 \le \phi < 110, \\ 1.65 - 0.01(\phi - 110), & 110 \le \phi < 140, \\ 1.35 - 0.005(\phi - 140), & 140 \le \phi \le 180. \end{cases} \tag{38}$$

The optimal trim $\tau^\star(\phi, \kappa)$ is computed by linear interpolation over mode-specific trim points. Trim efficiency is Gaussian:

$$e_\tau = \exp\left[-\frac{1}{2} \left(\frac{|\tau - \tau^\star|}{\sigma_\kappa} \right)^2 \right], \qquad \sigma_F = 0.12, \quad \sigma_D = 0.16. \tag{39}$$

The trim-induced speed multiplier is

$$m_\tau(e_\tau, \kappa) = \begin{cases} 0.60 + 0.65 e_\tau, & \kappa = F, \\ 0.72 + 0.42 e_\tau, & \kappa = D. \end{cases} \tag{40}$$

The target speed is

$$v_t^{\text{target}} = \min(v_t^{\text{base}} m_\tau(e_\tau, \kappa_t), 50). \tag{41}$$

The effective speed follows an inertia update:

$$v_{t+1} = I_{\kappa_t} v_t + (1 - I_{\kappa_t}) v_t^{\text{target}}, \qquad I_F = 0.98, \quad I_D = 0.85. \tag{42}$$

The boat enters foiling mode when $v_{t+1} \ge 18$ knots and exits when $v_{t+1} < 15$ knots.

3.6 Reward Modeling

The reward is dense and multi-objective. Let g_t be the active target, p_t the boat position, and

$$d_t = \|p_t - g_t\|_2, \qquad \Delta d_t = d_{t-1} - d_t. \tag{43}$$

The progress reward is asymmetric: losing distance is penalized more severely than gaining distance is rewarded. With $c_t = 1.8$ in foiling mode and $c_t = 0.65$ otherwise,

$$r_t^{\text{prog}} = \begin{cases} c_t \Delta d_t, & \Delta d_t \ge 0, \\ 1.8 c_t \Delta d_t, & \Delta d_t < 0. \end{cases} \tag{44}$$

VMG is the projection of the boat speed in the direction of the active target:

$$\text{VMG}_t = v_t \begin{bmatrix} \cos\psi_t \\ \sin\psi_t \end{bmatrix}^\top \frac{g_t - p_t}{\|g_t - p_t\|_2}, \qquad \widehat{\text{VMG}}_t = \text{clip}\left(\frac{\text{VMG}_t}{50}, -1, 1 \right). \tag{45}$$

With $\lambda_t = 1.6$ on the upwind leg and $\lambda_t = 1.25$ otherwise, the VMG term is

$$r_t^{\text{vmg}} = 8.5\lambda_t \max(\widehat{\text{VMG}}_t, 0) + 4.0\lambda_t \min(\widehat{\text{VMG}}_t, 0). \tag{46}$$

Let β_t be the bearing from the boat to the target and

$$h_t = \frac{|\text{wrap}(\beta_t - \psi_t)|}{\pi}. \tag{47}$$

The heading alignment term is

$$r_t^{\text{head}} = 1.2(1 - h_t) - 3.0 \max(h_t - 0.75, 0). \tag{48}$$

The rudder penalty discourages aggressive and oscillatory steering:

$$r_t^{\text{rudder}} = -2|u_t|^3 - 5(u_t - u_{t-1})^2. \tag{49}$$

The trim and foil rewards are

$$r_t^{\text{trim}} = 8.5(e_{\tau,t} - 0.72) - 5.5|\tau_t - \tau_{t-1}|^{1.5} - 18 \max(|\tau_t - \tau_t^\star| - \xi_t, 0)\mathbb{I}[v_t > 10], \tag{50}$$

$$r_t^{\text{vmg-trim}} = 5 \max(\widehat{\text{VMG}}_t, 0)(e_{\tau,t} - 0.50), \tag{51}$$

where $\xi_t = 0.18$ on the upwind leg and $\xi_t = 0.24$ otherwise. If a boat drops from foiling to displacement mode, it receives

$$r_t^{\text{drop}} = -40. \tag{52}$$

The residual speed and foil terms are

$$r_t^{\text{foil}} = \begin{cases} 1.1(v_t - 15), & \kappa_t = F, \\ -1.5, & \kappa_t = D, \end{cases} \qquad r_t^{\text{low}} = -3\mathbb{I}[v_t < 15], \tag{53}$$

and

$$r_t^{\text{speed}} = -0.35 \max(v^{\text{leg}} - v_t, 0) + 0.28 v_t \mathbb{I}[\Delta d_t > 0], \tag{54}$$

where $v^{\text{leg}} = 20$ knots on the upwind leg and 24 knots otherwise.

The multi-agent tactical advantage compares the two vessels' distances to their respective active targets:

$$r_t^{\text{tac}} = 1.5 \,\text{clip}\left(\frac{d_t^{\text{opp}} - d_t}{\sqrt{W^2 + L^2}}, -1, 1\right). \tag{55}$$

The course direction shaping term rewards movement in the appropriate vertical direction:

$$r_t^{y} = \begin{cases} 0.15(y_t - y_{t-1}), & \ell_t \in \{0, 1\}, \\ -0.15(y_t - y_{t-1}), & \ell_t \notin \{0, 1\}. \end{cases} \tag{56}$$

Collision avoidance is implemented through near-miss, time-to-collision, and direct collision penalties. If d_t^{ij} is the distance between the two vessels and $d_t^{ij} < 40$, the near-miss penalty is

$$p_t^{\text{near}} = 3\frac{40 - d_t^{ij}}{40}. \tag{57}$$

If the closing speed c_t^{ij} is positive, the time to collision is

$$\text{TTC}_t = \frac{d_t^{ij}}{c_t^{ij}}. \tag{58}$$

For $\text{TTC}_t < 4$, the predictive penalty is

$$p_t^{\text{ttc}} = 5\left(1 - \frac{\text{TTC}_t}{4}\right)\frac{40 - d_t^{ij}}{40}. \tag{59}$$

If $d_t^{ij} < 20$, the overlap penalty is

$$p_t^{\text{coll}} = 20\frac{20 - d_t^{ij}}{20}, \tag{60}$$

with a multiplier of 1.6 applied to the port-tack boat on opposite tacks. Severe violations such as hard collision, missed gate, out of bounds, wrong course movement, failed rounding, and rotation violation receive a terminal penalty of -10000.

Gate bonuses are sparse but important:

$$b_t = 400\,\mathbb{I}[\text{valid start gate}] + 500\,\mathbb{I}[\text{valid top rounding}] + B_t^{\text{finish}}, \tag{61}$$

where

$$B_t^{\text{finish}} = \begin{cases} 2500 + 1000\frac{1800-t}{1800}, & \text{first to cross the finish,} \\ 700, & \text{second to cross the finish,} \\ 0, & \text{otherwise.} \end{cases} \tag{62}$$

Combining the dense components, gate events, and terminal events, the step reward is

$$\begin{aligned} r_t = {} & r_t^{\text{prog}} + r_t^{\text{vmg}} + r_t^{\text{head}} + r_t^{\text{rudder}} + r_t^{\text{trim}} + r_t^{\text{vmg-trim}} + r_t^{\text{foil}} + r_t^{\text{low}} + r_t^{\text{speed}} \\ & + r_t^{\text{tac}} + r_t^{y} - 0.25 - p_t^{\text{near}} - p_t^{\text{ttc}} - p_t^{\text{coll}} + b_t + p_t^{\text{terminal}}. \end{aligned} \tag{63}$$

4 Experimental Setup and Results

4.1 Training Infrastructure

The training pipeline uses PPO [2] from Stable-Baselines3 [4]. The PettingZoo environment [3] is wrapped with SuperSuit, converted to a vectorized environment, monitored with `VecMonitor`, and augmented with `VecFrameStack`. The configuration is read from `config.yaml`; therefore the hyperparameters in Table 2 reflect the actual project configuration.

Parameter	**Configured value**
Algorithm	PPO
Total configured training steps	2,000,000
Parallel environments (`n_envs`)	16
Agents per environment	2
Frame stacking	4 frames
Effective input size	$23 \times 4 = 92$
Network architecture	MLP $[256, 256]$ for policy and value networks
Learning rate	2×10^{-4}
Rollout steps (`n_steps`)	4096
Batch size	1024
Optimization epochs	10
Discount factor γ	0.99
GAE parameter λ	0.95
PPO clipping range	0.2
Entropy coefficient	0.01
Success window size	100 episodes
Success threshold	0.95

Table 2: Training configuration used.

4.2 Available Evidence

The repository contains a trained model artifact, exported plots, and a rendered demonstration video, but does not contain raw evaluation logs, CSV summaries, or TensorBoard event files. Consequently, the report does not present tables of measured arrival rates or DNF percentages (these are only shown in the console during training).

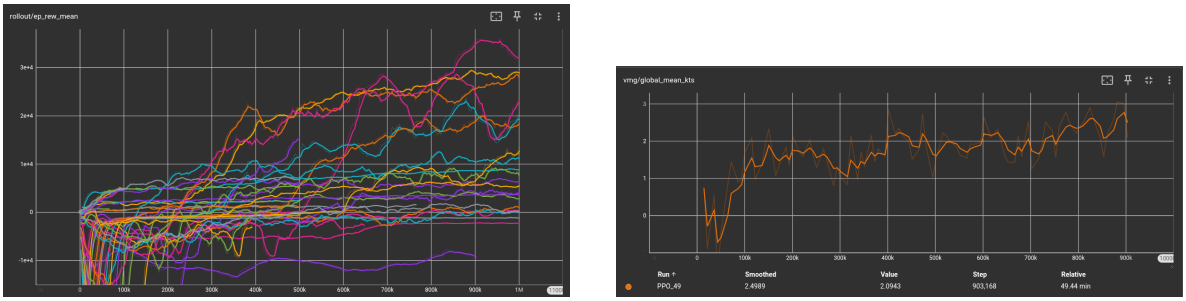

Figure 1: Available training plots. The left plot reports mean episodic reward; the right plot reports global mean VMG in knots. These figures support a qualitative discussion of learning dynamics.

The available curves indicate that the optimization process produced a policy with improved reward and VMG behavior over the course of training. The rendered trajectory also shows that the learned policy is capable of generating non-trivial sailing behaviors along the course, including gate-oriented navigation and opponent interaction.

4.3 Comparison with the Course Notebook

The course notebook introduces a didactic single-agent sailing task with a simplified state representation, basic actions, reward based on distance to target and speed, and a simplified polar/VMG model. The present project preserves the same conceptual core – learning a navigation policy from interaction – but substantially increases the environment complexity. It introduces two simultaneous agents, opponent-relative observations, PettingZoo parallel execution, continuous rudder and sail trim actions, stochastic wind, gate logic, collision handling, Rule 10-inspired penalties, and a hybrid displacement/foil performance model.

5 Conclusions and Future Work

5.1 Conclusions

This project started from a simple idea: transforming the course's didactic environment into something closer to a real regatta. The result is a simulator that, while preserving the conceptual structure of reinforcement learning presented in the lectures, introduces a series of mechanisms that make the problem substantially richer: two vessels competing simultaneously, a wind field that changes continuously, boats that can fly on foils or sail in displacement mode, and an articulated course with mandatory gates and right-of-way rules.

With continuous observations and actions and a dense, multi-objective reward function, tabular methods are not applicable, and PPO offers a good compromise between stability and the ability to handle continuous control spaces. The training plots show that the optimization process worked: mean episodic reward grows over the course of training, and so does mean VMG, suggesting that the boats have learned something non-trivial about how to navigate efficiently.

Finally, looking at the rendered trajectories, what we can state is that the learned policy produces gate-oriented navigation, opponent interaction, and foil transitions under appropriate conditions. That is not a trivial result for an environment of this complexity.

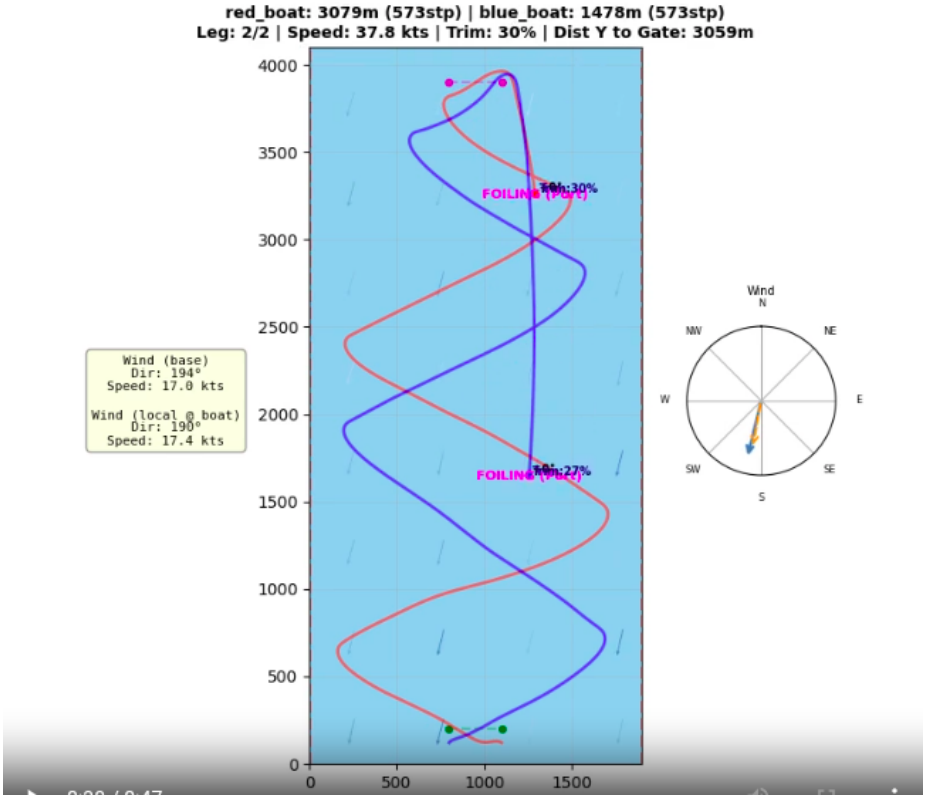

Figure 2: Rendered simulation frame from the available demonstration. The visualization is used as qualitative evidence of the learned navigation behavior, not as a statistical evaluation over multiple seeds.

5.2 Future Work

5.2.1 Pre-Start Phase

One of the most interesting developments that could be integrated into the environment concerns the simulation of the pre-start phase, that is, the period preceding the official race start. In real match races, this phase is anything but passive: vessels maneuver actively near the starting line seeking the most advantageous position, managing timing with precision, and trying to put pressure on the opponent without incurring a premature start (OCS, *On Course Side*).

The idea would be to introduce a dedicated state variable that distinguishes the pre-start phase from the actual race. During the pre-start, boats would be spawned below the starting line and would maneuver freely, receiving different reward signals from those of the race: they would be rewarded for proximity to the line, for dynamic movement, and for their ability to position themselves favorably relative to the opponent. Crossing the line prematurely would incur an immediate penalty and episode termination, replicating the OCS penalty of real regattas.

After a fixed number of steps, the environment would automatically transition to the competitive phase, activating the full race logic with gates, rounding, and finish. This clear separation between the two phases would allow agents to learn pre-start tactical behaviors distinct from course navigation behaviors, considerably enriching the complexity and realism of the simulation.

5.2.2 Collection and Analysis of Evaluation Data

A concrete limitation of the current work is the absence of systematic evaluation data. During training, mean episodic reward and VMG are recorded, but no tracking is kept of how episodes terminate. Adding a dedicated evaluation script that runs a fixed number of episodes on controlled seeds and records the termination reason for each one would allow a much more precise understanding of what the policy is actually learning.

The most useful data to collect would be the course completion rate, that is, the percentage of episodes in which at least one of the two boats crosses the finish line, and the distribution of outcome categories for episodes that do not reach the end: how many terminate due to `timeout`, how many due to `missed_top_gate` or `missed_bottom_gate`, how many due to `collision`, how many due to `wrong_course` or `spin_violation`. This distribution is particularly informative because the different failure categories indicate qualitatively different problems in the policy: a high `timeout` rate would suggest that the boats are sailing but in too slow or dispersive a manner, while a high `missed_top_gate` rate indicates that upwind rounding has not yet been correctly learned.

Having these numbers exported in a reproducible CSV table would also allow rigorous comparison of different model versions, instead of relying exclusively on qualitative observation of rendered trajectories.

References

[1] Francesco Pivi. *Introduction to Reinforcement Learning*. Course notes, 2025.

[2] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. *Proximal Policy Optimization Algorithms*. arXiv preprint arXiv:1707.06347, 2017.

[3] J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S. Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, and others. *PettingZoo: Gym for Multi-Agent Reinforcement Learning*. Advances in Neural Information Processing Systems, 2021.

[4] Antonin Raffin, Ashley Hill, Adam Gleave, Anssi Kanervisto, Maximilian Ernestus, Noah Dormann, and others. *Stable-Baselines3: Reliable Reinforcement Learning Implementations*. Journal of Machine Learning Research, 22(268):1–8, 2021.

[5] Andrej Karpathy. *Neural Networks: Zero to Hero*. Course playlist, 2022. URL: https://karpathy.ai/zero-to-hero.html.